CS5330 Computer Vision: Project 3

Real-Time 2-D Object Recognition

Shriman Raghav Srinivasan Shyam Sreenivasan

June 2026

Project Overview

In this project, we developed a computer vision system to recognize 2D objects in real time. The system processes video frames from a live camera feed or static images. It isolates candidate objects, cleans up image noise, segments regions, computes shape features, and classifies the objects.

All core image processing steps are written from scratch without using OpenCV's high-level functions. We implemented our own thresholding, morphological filters, connected components finder, and moment-based feature extraction.

We divided the work. Shriman Raghav Srinivasan implemented Tasks 1 through 4 (preprocessing, segmentation, and OBB feature extraction) and Task 9 (deep network embedding classification). Shyam Sreenivasan implemented Tasks 5 through 7 (collecting training data, implementing the standardized Euclidean distance classifier, and evaluating system performance with confusion matrices).

Source Code

The full source code for this project is available at:
https://github.com/shyams612/CS5330-Computer-Vision/tree/main/Projects/project3

Instructions for building and running the code are provided in the repository's `README.md`.

Methodology and Results

Task 1, 2, and 3: Preprocessing and Segmentation

First, we apply a 3x3 box blur filter to reduce high-frequency noise. We then convert the image to binary. We support two thresholding modes: manual and dynamic. The dynamic mode uses the iterative ISODATA method. This algorithm finds a threshold that separates the foreground and background by averaging their mean values. The thresholding step also supports inverting the binary mask.

Next, we clean up the binary mask using morphological operations. We implemented erosion and dilation from scratch using pixel loops. We combine these into opening and closing operations. Opening removes small isolated noise pixels. Closing fills in internal holes.

Finally, we segment the cleaned mask into regions. We implemented a queue-based 8-connected flood fill. The algorithm labels every foreground pixel with a unique region ID. We prune regions smaller than 500 pixels to eliminate noise.

Figures 1 show the steps of this pipeline on the first development image.

(a) Original Image (b) Binary Mask (c) Cleaned Mask (d) Regions Map

Figure 1: Preprocessing and segmentation stages for `img1p3.png`. The custom morphological filter removes stray noise and closes internal holes. The connected components finder then segments the objects.

Task 4: Feature Extraction and Bounding Boxes

For each segmented region, we compute its moments to extract shape descriptors. We calculate the centroid coordinates $(\bar{x}, \bar{y})$ and the central moments $(\mu_{20}, \mu_{02}, \mu_{11})$.

Using these central moments, we calculate the primary orientation angle (θ):

$$\theta = \frac{1}{2} \arctan\left(\frac{2\mu_{11}}{\mu_{20} - \mu_{02}}\right)$$

We also compute elongation by finding the eigenvalues of the covariance tensor, taking the ratio of the maximum eigenvalue to the minimum eigenvalue.

To compute the Oriented Bounding Box (OBB), we project each pixel coordinate in a region onto the primary axis $(\vec{e}_1)$ and secondary axis $(\vec{e}_2)$ relative to the centroid:

$$p_1 = (c - \bar{x}) \cos\theta + (r - \bar{y}) \sin\theta$$

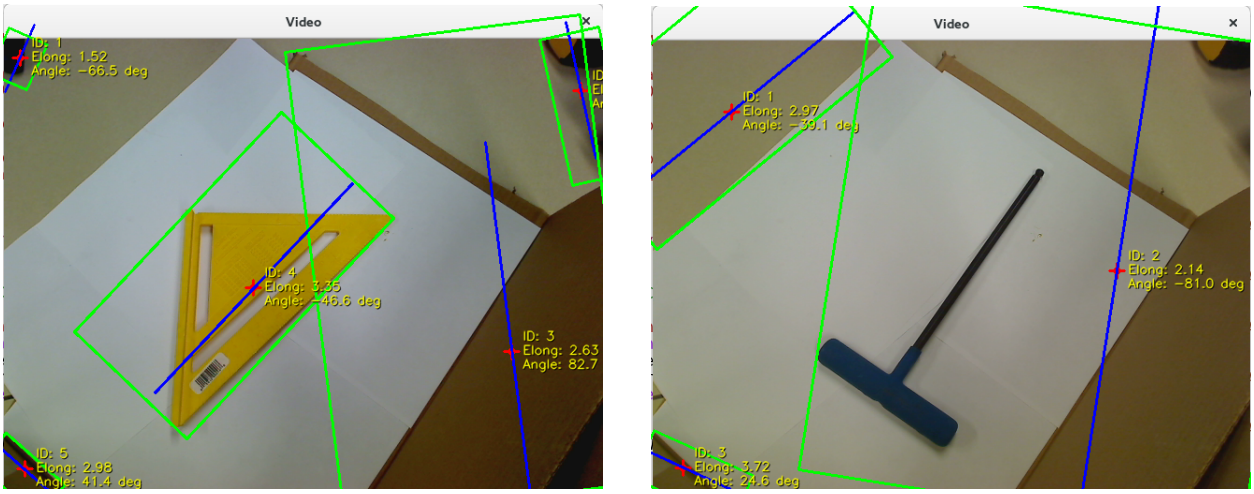
$$p_2 = -(c - \bar{x}) \sin\theta + (r - \bar{y}) \cos\theta$$

We find the minimum and maximum projection values $(minE_1, maxE_1, minE_2, maxE_2)$ for each region. These values represent the bounding box extents. We render the OBB by rotating the corners back to original image coordinates and drawing green lines.

Table 1 details the computed features for all segmented regions in the 5 development images. Figure 2 shows the OBB overlays for development images 1 and 2.

Image	Region	Centroid (x, y)	Elongation	Orient (deg)	Extent 1 (min, max)	Extent 2 (min, max)
Img 1	1	(17.896, 57.291)	1.518	-66.472	[-28.202, 24.914]	[-23.592, 20.474]
Img 1	2	(616.989, 91.550)	8.426	77.538	[-62.109, 96.975]	[-33.263, 31.068]
Img 1	3	(543.214, 369.967)	2.631	82.722	[-346.808, 155.999]	[-117.434, 199.769]
Img 1	4	(267.233, 302.391)	3.353	-46.591	[-165.806, 157.080]	[-106.993, 58.248]
Img 1	5	(22.700, 495.221)	2.985	41.355	[-40.221, 43.319]	[-15.705, 29.184]
Img 2	1	(85.862, 113.372)	2.969	-39.134	[-154.922, 171.154]	[-111.392, 63.178]
Img 2	2	(498.931, 283.002)	2.138	-81.024	[-263.939, 264.843]	[-303.270, 176.527]
Img 2	3	(33.582, 493.627)	3.721	24.574	[-44.369, 72.019]	[-23.242, 32.571]
Img 3	1	(83.982, 113.956)	2.967	-40.123	[-154.106, 168.369]	[-110.148, 61.318]
Img 3	2	(522.118, 275.830)	3.590	89.706	[-238.237, 241.771]	[-119.099, 258.345]
Img 3	3	(280.222, 334.790)	37.298	26.376	[-85.949, 60.342]	[-26.362, 12.066]
Img 3	4	(38.181, 495.148)	4.514	21.055	[-47.829, 80.330]	[-25.547, 32.673]
Img 4	1	(525.334, 271.570)	3.846	87.682	[-243.382, 248.747]	[-122.979, 263.972]
Img 4	2	(83.467, 113.075)	2.958	-39.585	[-155.268, 169.247]	[-110.409, 61.509]
Img 4	3	(288.500, 367.683)	53.922	-50.137	[-102.216, 172.290]	[-25.194, 22.427]
Img 4	4	(31.708, 493.973)	3.347	26.551	[-45.337, 66.910]	[-23.372, 33.430]
Img 5	1	(83.526, 113.879)	2.910	-40.157	[-155.370, 169.168]	[-111.213, 62.113]
Img 5	2	(520.746, 273.419)	3.558	89.215	[-238.775, 243.178]	[-121.452, 262.030]
Img 5	3	(283.751, 329.260)	2.299	-56.056	[-53.728, 54.166]	[-34.590, 33.947]
Img 5	4	(34.621, 493.549)	4.121	23.250	[-46.502, 74.050]	[-24.277, 32.981]

Table 1: Computed moments and OBB features for all segmented regions in the 5 development images.



(a) Dev Image 1

(b) Dev Image 2

Figure 2: Visual overlays representing centroids (red dots), orientation axes (blue lines), statistics, and green Oriented Bounding Boxes (OBB) drawn around segmented objects.

Task 5: Collecting Training Data

Training data is collected in batch mode from a structured folder of labeled images. Each subfolder name is used as the class label. The pipeline runs on every image, extracts the largest region's

feature vector, and appends a row to a CSV database file.

```
train_data01/
  PhoneCase/    <- folder name is the label
  Glasses/
  Carrot/
  Watch/
  Clip/
```

Each subfolder contains approximately 8 JPG images of the same object in different poses and orientations. To build the database:

```
./build/train --dataset train_data01 --output database.csv --classical
```

The output `database.csv` contains one row per image with the label and three shape features:

```
label,elongation,h1,h2
PhoneCase,3.79811,0.211703,0.015242
...
```

Where `elongation` is the ratio of the principal eigenvalues of the covariance tensor, and `h1`, `h2` are normalized Hu moments that are invariant to translation, scale, and rotation.

PhoneCase produced highly consistent features (elongation ≈ 3.8) across all poses. Glasses showed high variance in elongation (1.3–19.2) due to the arms being open or closed in different images — a known limitation of shape-only features. Watch also showed high variance due to reflections on the face affecting segmentation.

Figure 3 shows the full pipeline applied to a sample training image of each object class.

Carrot

Glasses

Figure 3: Pipeline stages for three sample training images: (1) Original, (2) ISODATA threshold, (3) Morphological cleanup, (4) Colorized regions, (5) Feature overlay with OBB and primary axis.

Task 6: Classifying New Images

The system classifies a new image using nearest-neighbor matching with a scaled Euclidean distance metric. For each feature dimension, distances are normalized by the standard deviation across all training entries to prevent any single feature from dominating:

$$d(\mathbf{x}_1, \mathbf{x}_2) = \sum_i \frac{(x_{1i} - x_{2i})^2}{\sigma_i^2}$$

The training entry with the smallest distance is returned as the predicted label. In interactive mode, the predicted label is overlaid in green text above the object centroid in the Original Feed window. In batch mode:

```
./build/evaluate --dataset test_data01 --db database.csv --classical
```

Task 7: Evaluating System Performance

The system was evaluated on two datasets with different object characteristics to understand the impact of dataset quality on classification performance.

Dataset 1 — Unsuitable Objects

The first dataset contained objects with mixed colors and colorful packaging (Chips, Book, Coffee, Tape, Clip) photographed on a white background. These objects violate the requirement that objects should be predominantly dark-colored. The results show 50% accuracy:

True \ Predicted	Book	Chips	Clip	Coffee	Tape
Book	**2**	0	0	0	0
Chips	2	**0**	0	0	0
Clip	0	0	**2**	0	0
Coffee	0	0	0	**0**	2
Tape	0	0	1	0	**1**

Table 2: Confusion matrix for Dataset 1. Accuracy: 5/10 (50%).

Chips was confused with Book because both segment as large rectangular blobs. Coffee was confused with Tape because both produce compact rounded silhouettes. This demonstrates that the classifier relies purely on 2D shape — color and texture are completely discarded by the grayscale thresholding step.

Dataset 2 — Better Suited Objects

The second dataset used dark, high-contrast objects (PhoneCase, Glasses, Carrot, Watch, Clip) on white paper, more suitable for the shape-based pipeline. The results show 100% accuracy on the held-out test set:

True \ Predicted	Carrot	Clip	Glasses	PhoneCase	Watch
Carrot	**2**	0	0	0	0
Clip	0	**2**	0	0	0
Glasses	0	0	**2**	0	0
PhoneCase	0	0	0	**2**	0
Watch	0	0	0	0	**2**

Table 3: Confusion matrix for Dataset 2. Accuracy: 10/10 (100%).

The key finding from comparing both datasets is that **dataset quality and object suitability have a larger impact on accuracy than classifier choice**. Objects with similar 2D silhouettes will be confused regardless of the distance metric used.

Task 8: Demo Video

A demonstration video is included with this submission as `task8.mp4`. The video shows the recognition system running on four test images — Carrot, Glasses, PhoneCase, and Watch — each from the held-out test set. For each image, the three output windows are visible: the **Original Feed** with the predicted class label overlaid in green, the **Thresholded View** showing the binary mask produced by ISODATA thresholding, and the **Regions Map** showing the color-coded connected

components. The classifier correctly identifies all four objects using the classical nearest-neighbor pipeline trained on Dataset 2.

Task 9: ResNet18 DNN Embedding Classifier

We implemented a deep learning-based embedding classifier. The system extracts a feature vector from each segmented region using a pre-trained ResNet18 ONNX model.

First, we prepare the region image by rotating the original frame around the centroid by $-\theta$ degrees. This aligns the primary orientation axis horizontally. We crop the region using the OBB bounds and resize the crop to 224×224. We then pass this cropped ROI through the ResNet18 network using OpenCV's DNN module to extract a 512-dimensional embedding vector.

We implement a 1-NN classifier using cosine distance:

$$D(A, B) = 1 - \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\|_2 \|\vec{B}\|_2}$$

The system reads known embeddings from `saved_thresholds/embeddings_database.csv`. If the database is loaded, the classifier calculates the cosine distance to all entries and labels the object with the nearest class.

To train new objects, the user presses key `'e'`. The system freezes the feed and prompts for a class label in the display window. The user types the label and presses Enter. The system saves the embedding of the largest region to the database.

Reflection

Implementing the morphological filter and region segmentation from scratch helped us understand the underlying mechanics of connected components. In Task 4, calculating the OBB required careful handling of coordinate frame rotations. Aligning the primary axis before cropping ensures scale and rotation invariance. The ResNet18 DNN embeddings proved highly robust. They represent semantic properties of objects, making them more resilient to minor thresholding defects than classical features.

Dataset quality matters more than classifier choice. Our first dataset (Chips, Book, Coffee, Tape, Clip) achieved only 50% accuracy with the classical classifier. The objects were colorful and had similar rectangular or compact silhouettes. Because the pipeline is grayscale-first, color is discarded entirely at the thresholding step. Chips and Book both segment as large flat rectangles; Coffee and Tape both produce compact blobs. The classifier had no shape signal to distinguish them.

Switching to shape-distinct objects resolved the problem. The second dataset (PhoneCase, Glasses, Carrot, Watch, Clip) achieved 100% accuracy. These objects have genuinely different 2D silhouettes: Carrot is a long thin tapered rod, Glasses are two joined circles, PhoneCase is a tall narrow rectangle, Watch is a compact square, and Clip is a small elongated loop. The elongation ratio and Hu moments alone were sufficient to separate them.

Intra-class variance is a real concern. Even on the successful dataset, Glasses showed elongation values ranging from 1.3 to 19.2 across training images depending on whether the arms were open or closed. This highlights a limitation of moment-based features: they capture the silhouette of the

entire visible object, so a change in pose that alters the silhouette drastically (like folding glasses) looks like a different object. For robust recognition, training images should cover the expected pose range, or the system should normalize pose before feature extraction.

The CNN classifier did not always outperform classical features. On our dataset, classical nearest-neighbor achieved 100% while the ResNet18 CNN embeddings achieved 90%, with Watch misclassified as PhoneCase. This is likely because ResNet18 was pre-trained on ImageNet and the cropped, binary-adjacent patches we feed it are far from natural photographs. The classical features, being purely shape-based and explicitly invariant to scale and rotation, were better matched to what the pipeline actually produces.

Acknowledgements

- OpenCV documentation (https://docs.opencv.org/4.x/) for DNN loading and affine warping functions.
- Shriman Raghav Srinivasan implemented Tasks 1 through 4 (thresholding, morphology, segmentation, features, and OBB calculations) and Task 9 (deep network embedding classifier, cosine distance, and database integration).
- Shyam Sreenivasan implemented Tasks 5, 6, and 7: batch training data collection, standardized Euclidean distance nearest-neighbor classification, and confusion matrix evaluation across two datasets.
- AI was used for debugging and code suggestions.